Adversarial testing of network protocols

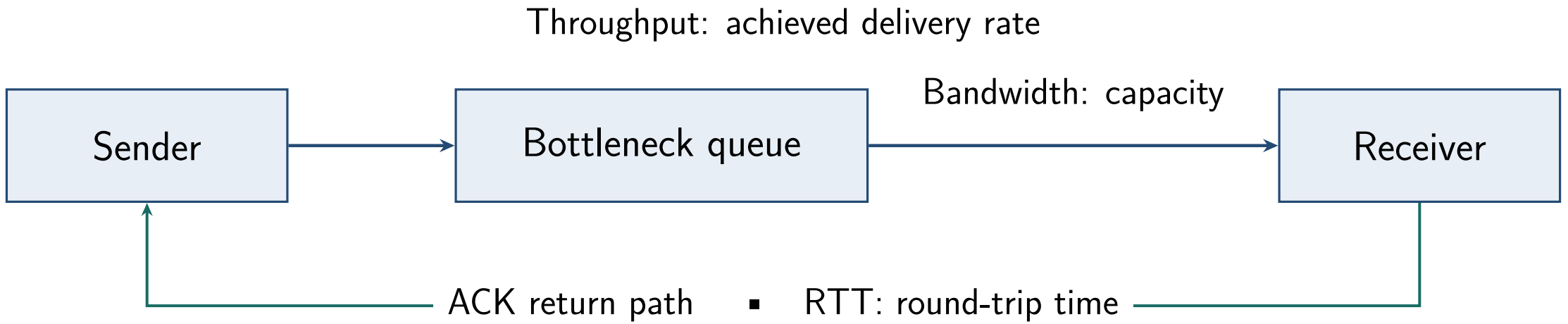
AdvNet

Revealing Performance Issues in Network Protocols
by Generating Adversarial Environments

Can we automatically find conditions where a protocol performs much worse?

Congestion control balances throughput and delay

Sending too slowly wastes capacity; sending too quickly builds queues.



Congestion window

Limits unacknowledged data in flight.

Pacing

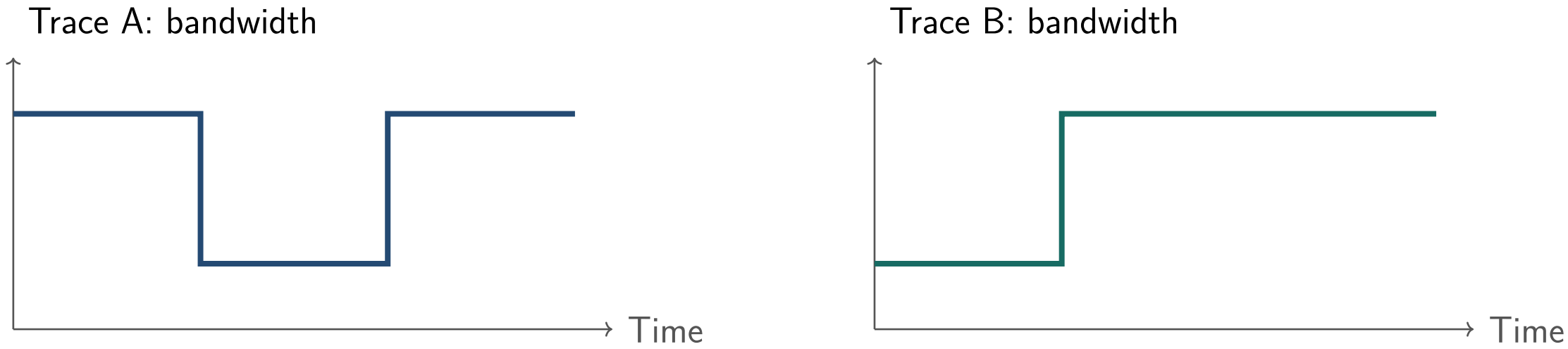
Controls when packets leave the sender.

Long RTT + high bandwidth requires enough data in flight.

Diagram by presenters.

The order of network changes matters

Protocols retain state while bandwidth and delay change.



Same average; different transitions

17 / 17 tested TCP algorithms had a vulnerable environment.

The task: search for these environments and investigate the resulting gaps.

Trace illustration by presenters.

Existing tests cover different parts of the problem

Where do test cases come from?

Hand-designed tests
Easy to interpret; coverage follows the designer's expectations.

Real network traces
Observed conditions; coverage follows collection sites and times.

What executes the test?

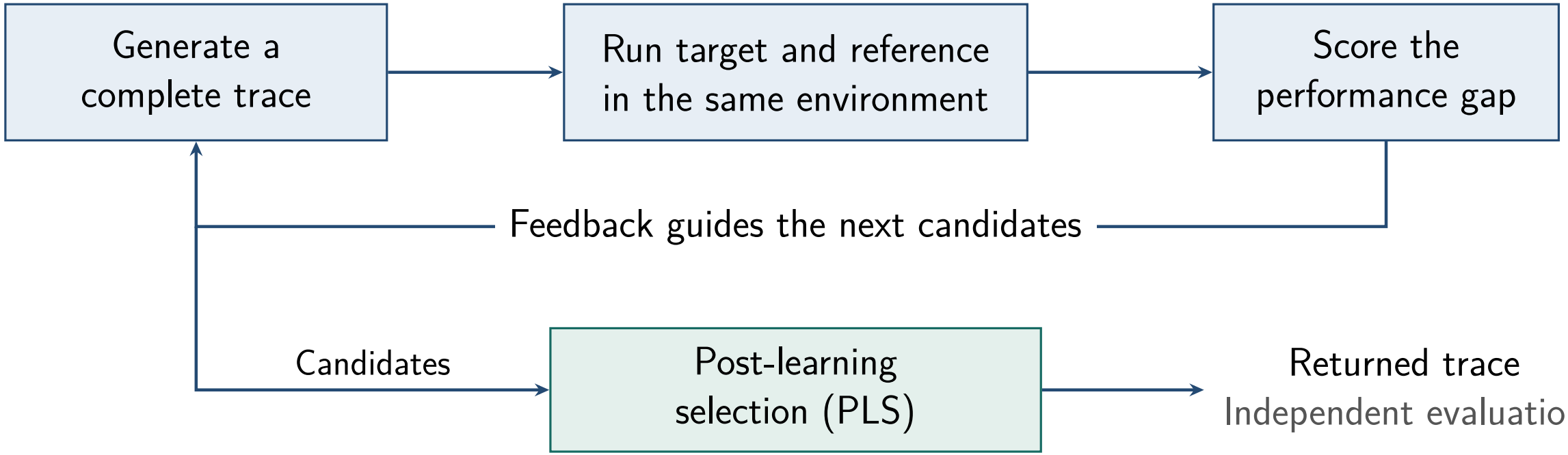
Simulation
Protocol models under modeled network conditions.

Emulation
Real protocol code under controlled conditions; runs cost time and vary.

Earlier adversarial testing: CC-Fuzz
AdvNet emphasizes real implementations and efficient environment search.

AdvNet searches over the environment

User inputs: parameter bounds, target, reference, performance metric



A larger score means a larger relative weakness in the target.

Architecture redrawn from Fig. 1.

Two challenges shape the design

1. Sample efficiency

- Each evaluation runs a real protocol.
- Long traces and repeated measurements consume the search budget.

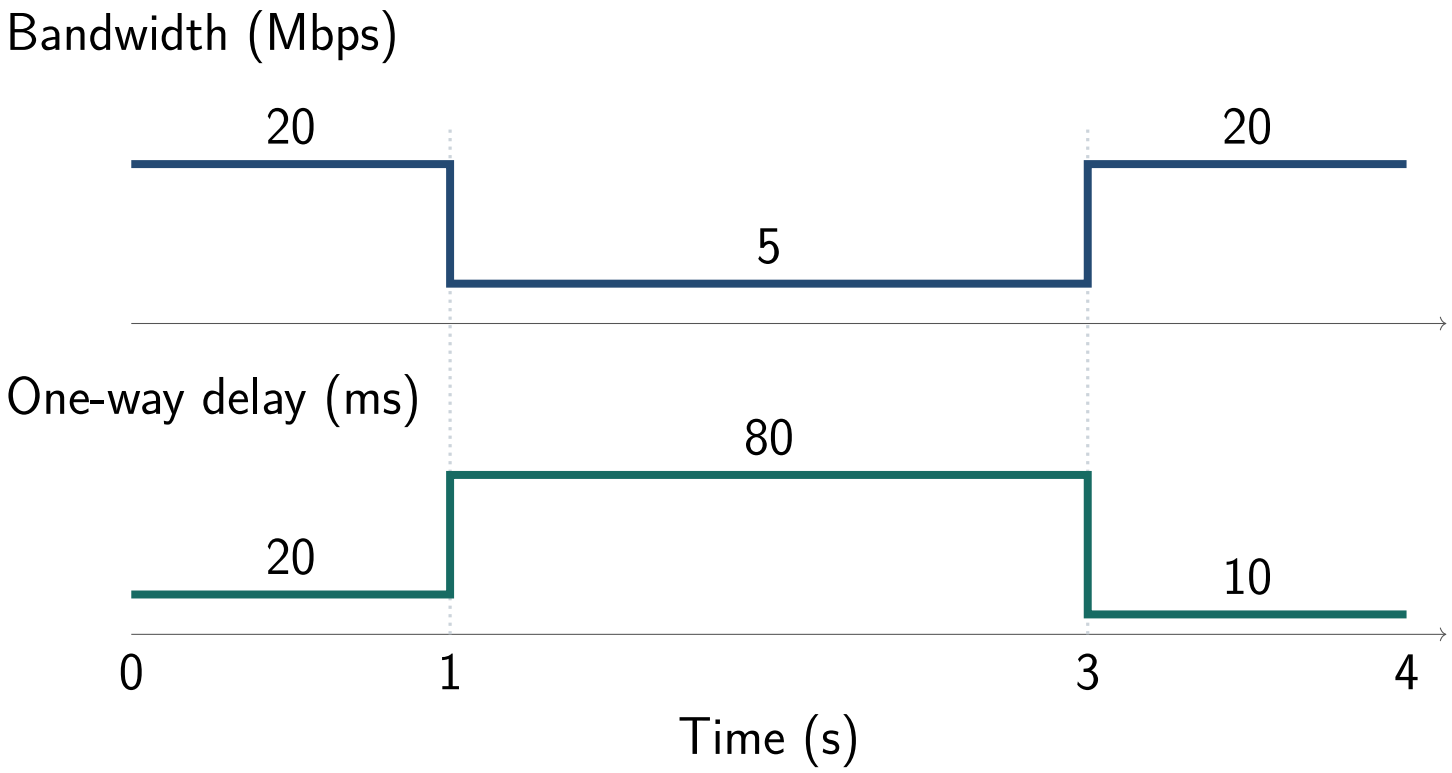
2. Practical usefulness

- Very low capacity can make every protocol perform poorly.
- A reference exposes a gap worth investigating.

Find strong candidates with few runs; make the comparison meaningful.

One candidate is a bounded network trace

$T = (B, L, D, I)$ Bandwidth, delay, duration, fixed settings



Three intervals

$D = (1, 2, 1)$ seconds

Fixed within a trace

I: queue capacity

Multipath

Each link has its own B, L; intervals share D.

Search bounds: 1–100 Mbps; 5–100 ms; 0.5–2.5 s per interval.

Trace illustration by presenters; parameter bounds from Table 1.

A compact representation improves the search

PDO: a packet delivery opportunity specifies when a packet can leave.

Target	Intervals	PDOs
BBR	**79.11%**	14.31%
CUBIC	**46.24%**	29.78%
Vegas	**84.71%**	27.29%
Reno	**41.26%**	28.80%

More PDOs, harder search

BBR, 25 → 100 PDOs:

14.31% → 5.91%

Why intervals help

Fewer dimensions

Direct control of delay changes

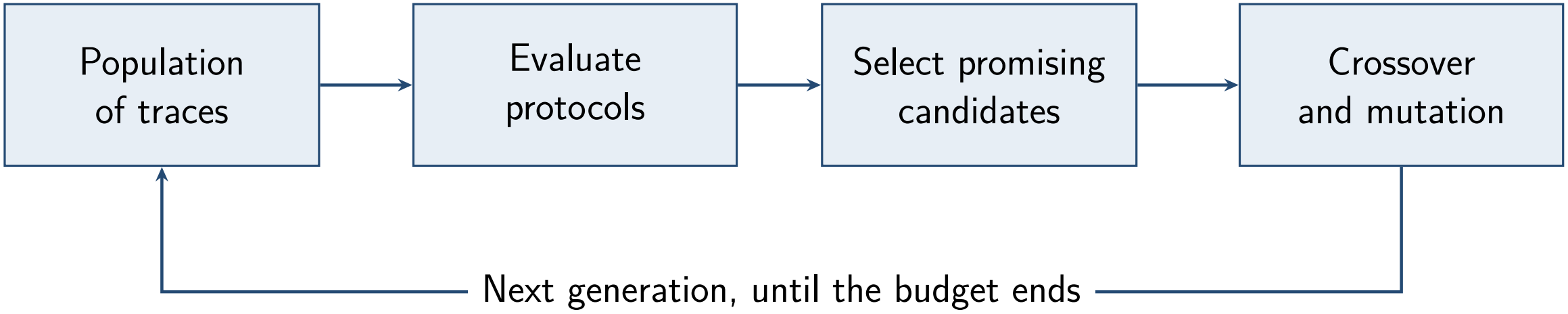
Same AdvNet framework and matched configurations.

Score: throughput shortfall relative to capacity. Higher is more adversarial.

Table 2 (reformatted); PDO dimension comparison from §4.2.1.

GA evolves a population of complete traces

Generate first, then replay: avoids synchronizing an online adversary with both target and reference during execution.



Genes: bandwidths, delays, durations, and fixed parameters.

Fitness = measured performance-gap score.

GA diagram by presenters.

Crossover recombines; mutation introduces new values

Two-point crossover

Exchange the middle segment.

	1	2	3	4
Parent A	20	5	40	10
Parent B	60	30	15	80
Child A	20	30	15	10
Child B	60	5	40	80

Uniform integer mutation

Each gene changes with probability p.

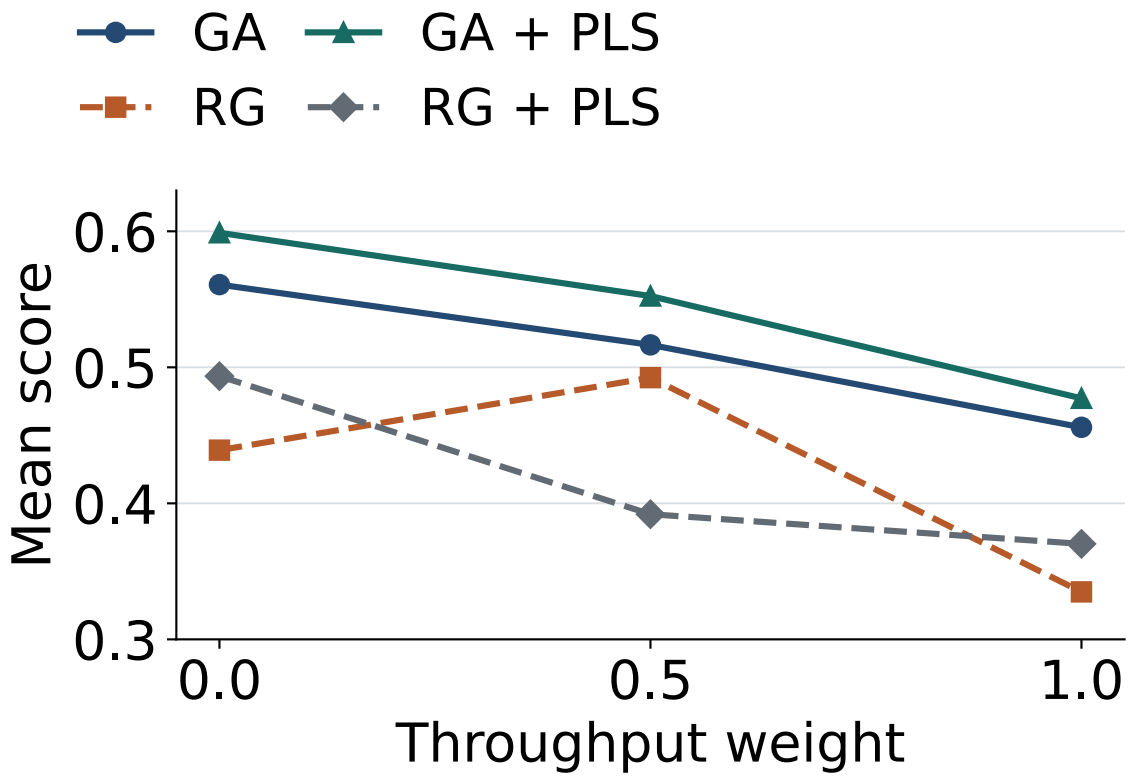
$$15 \rightarrow 73$$

Sample an integer from that gene's allowed range.

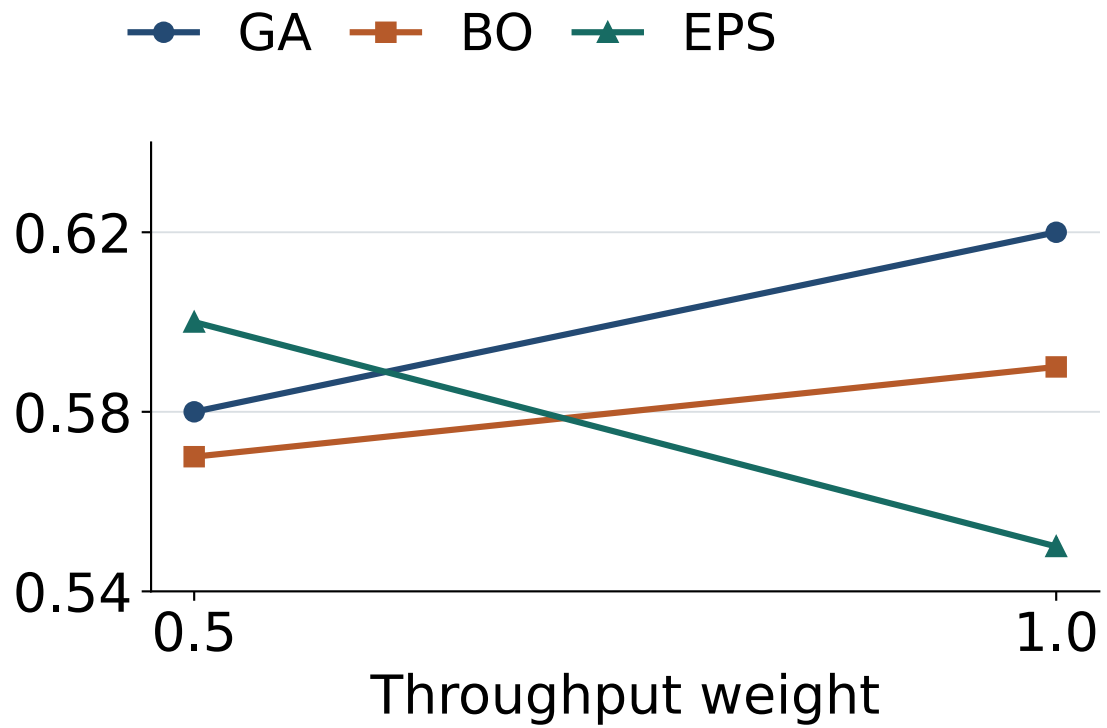
Here: 1–100.

Genetic search improves on random generation

One-hour budget: GA vs. random



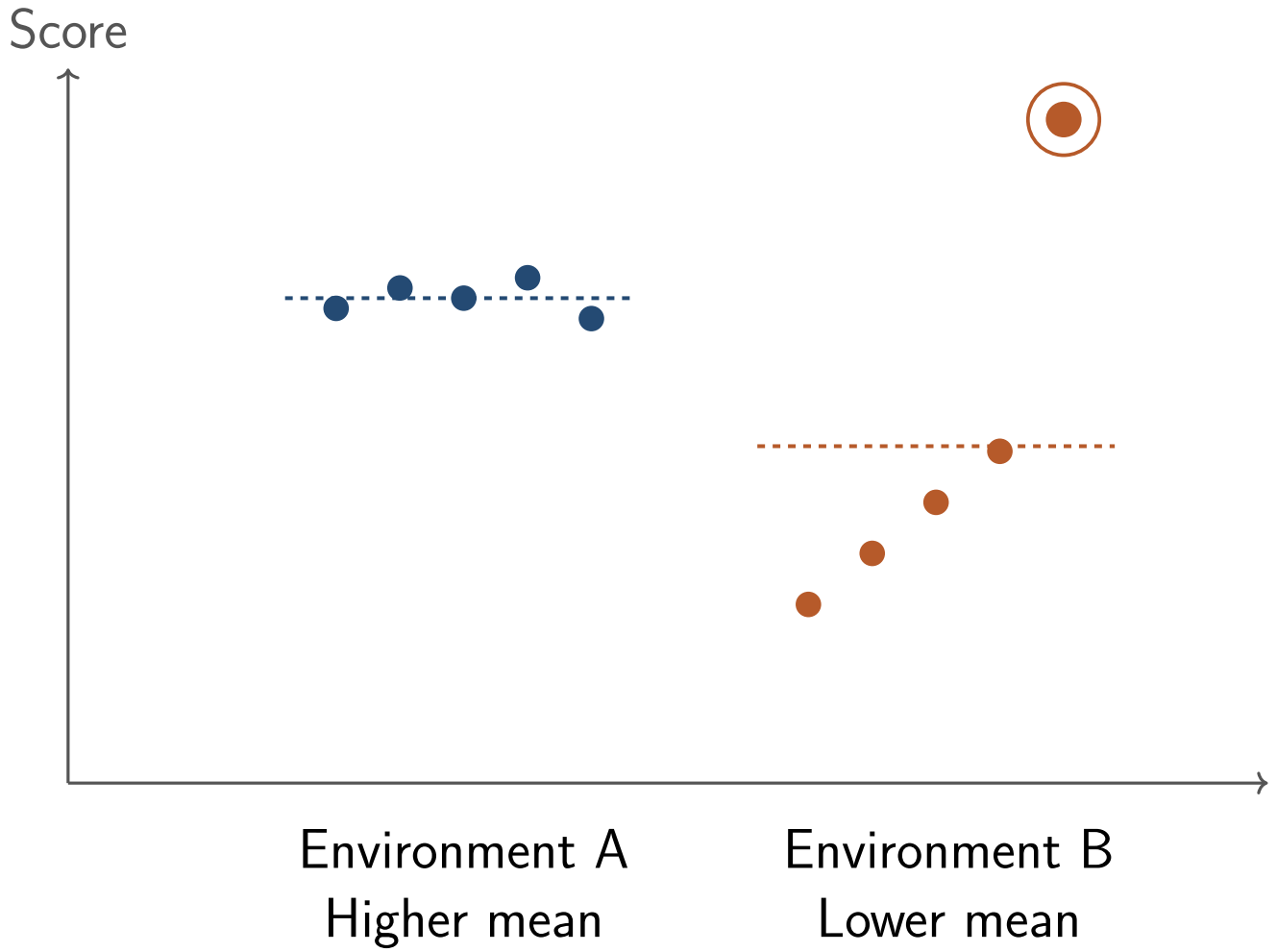
Two-hour budget: GA, BO, EPS



RG: random generation BO: Bayesian optimization EPS: epsilon-greedy

Fig. 3a–b (redrawn).

The highest observation can be a lucky outlier



SimpleMax

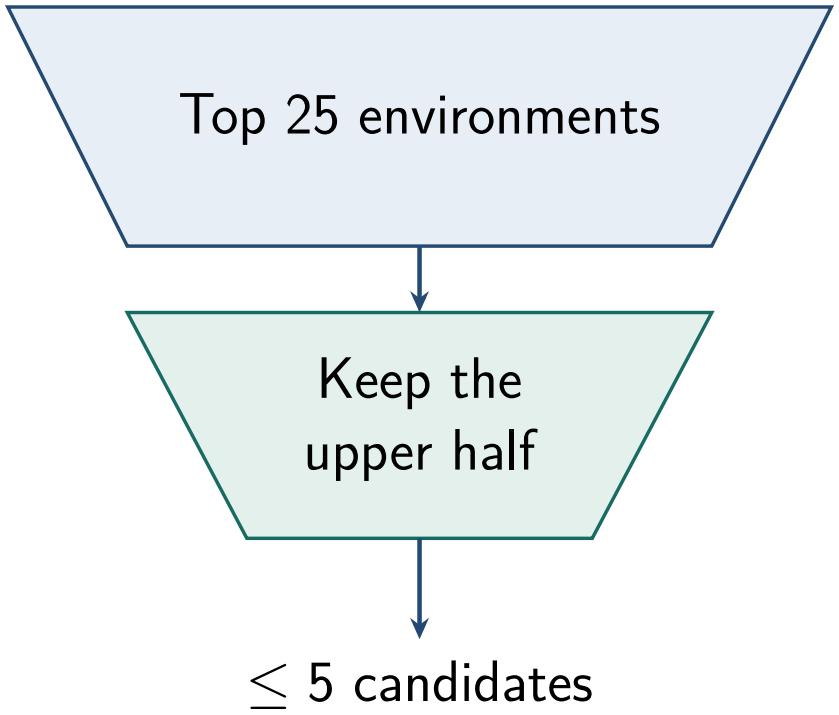
Return the trace with the largest score observed during search.

One unusually high run can make B look best.

Re-evaluation tests whether the high score persists.

Measurement illustration by presenters.

Post-learning selection focuses runs on strong candidates



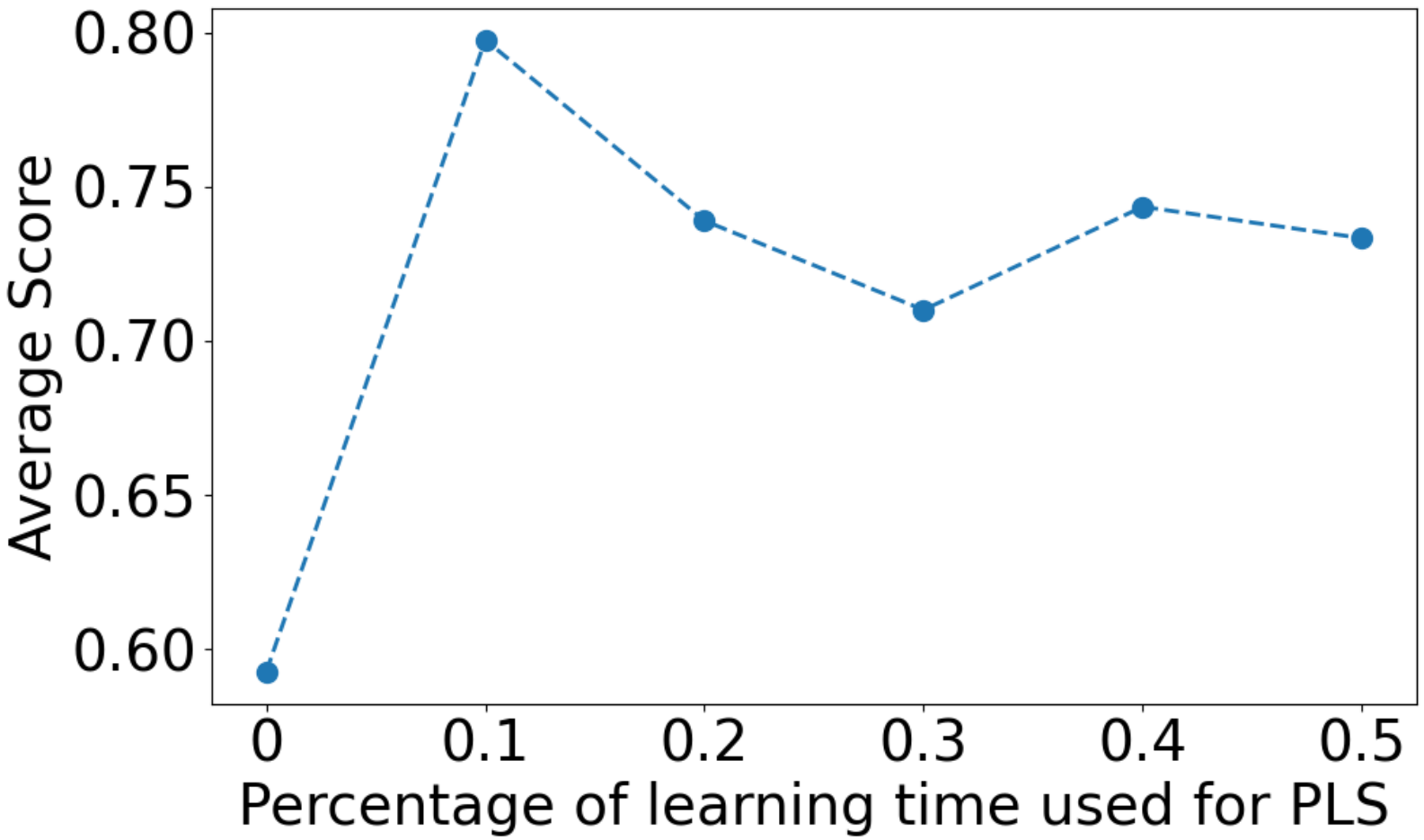
Multi-round elimination (MRE)

- Run each remaining candidate once per round.
- Rank by the mean observed so far; remove roughly half.
- At five or fewer, keep measuring until time is up; return the highest mean.

During search: medians → PLS: means → Independent evaluation

Selection diagram by presenters.

Reserve a small part of the budget for selection



90%

Search for environments

10%

Post-learning selection

Best split in this experiment.

Compact traces + GA + PLS

Fig. 2 (original).

A relative gap makes the objective useful

For a higher-is-better metric, with reference R and target T:

$$s = \frac{R - T}{\max(R, T)}$$

Reference	Target
10 Mbps	**2 Mbps**

$$\frac{10 - 2}{10} = \mathbf{0.8}$$

For a lower-is-better metric such as completion time, use $T - R$.

Higher score = larger target disadvantage. The score lies in $[-1, 1]$.

Choose which performance tradeoff to test

Normalize throughput τ and delay d so both terms are higher-is-better.

$$u = w \frac{\tau}{\tau_{\max}} + (1 - w) \frac{d_{\min}}{d}$$

$\tau_{\max}$: throughput bound $d_{\min}$: minimum delay w: throughput weight

$w = 0$	$w = 0.5$	$w = 1$
Delay only	Equal weighting	Throughput only

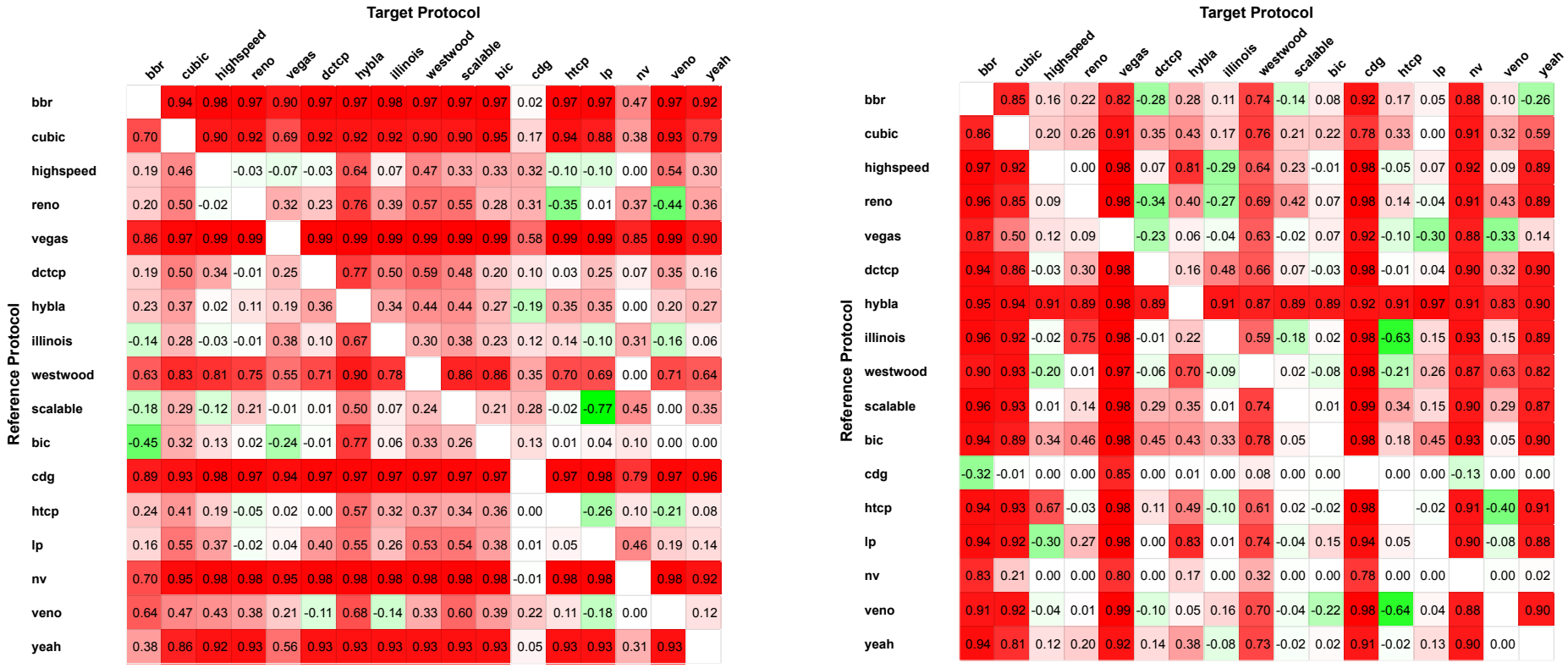
Compute the relative gap between reference and target utilities.

Change the reference to study weaknesses, regressions, or design goals.

Every tested protocol has a vulnerable environment

17 protocols $\times$ 16 references $\times$ 2 objectives $=$ 544 scenarios.

Adversarial environments found in 448 scenarios.



(a) $t_{\text{coeff}} = 0.5$ (b) $t_{\text{coeff}} = 1$

Fig. 4a–b (original vector figure).

BBR and CUBIC expose each other's blind spots

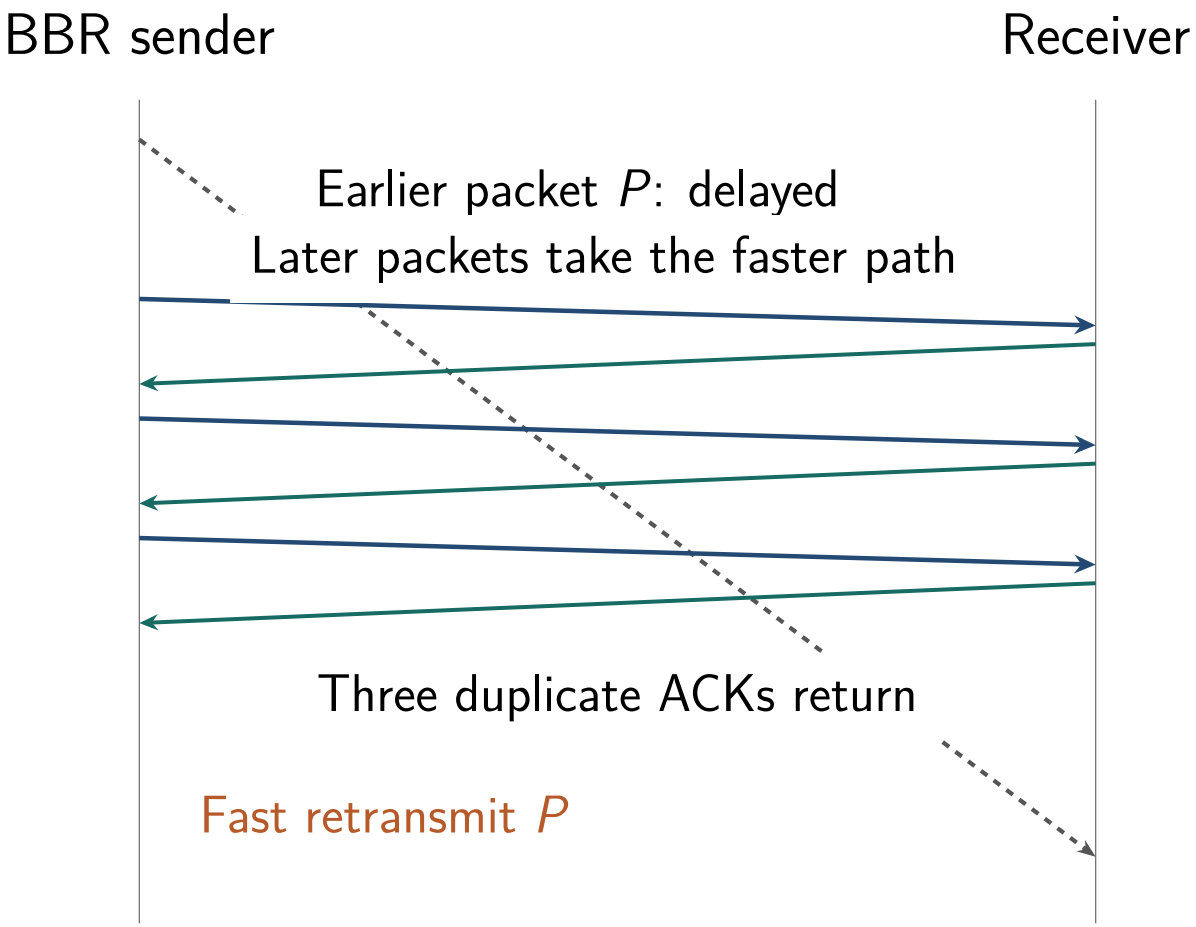
Reference	Target	50/50 utility	Throughput
BBR	CUBIC	**0.94**	**0.85**
CUBIC	BBR	**0.70**	**0.86**

Both directions reveal substantial gaps, in different environments.

The next step is to inspect the discovered traces and find the cause.

BBR/CUBIC entries from Fig. 4a–b (reformatted).

A sudden RTT drop reorders packets



1. High RTT, then a sharp drop

Later packets overtake the earlier packet.

2. A gap in sequence numbers

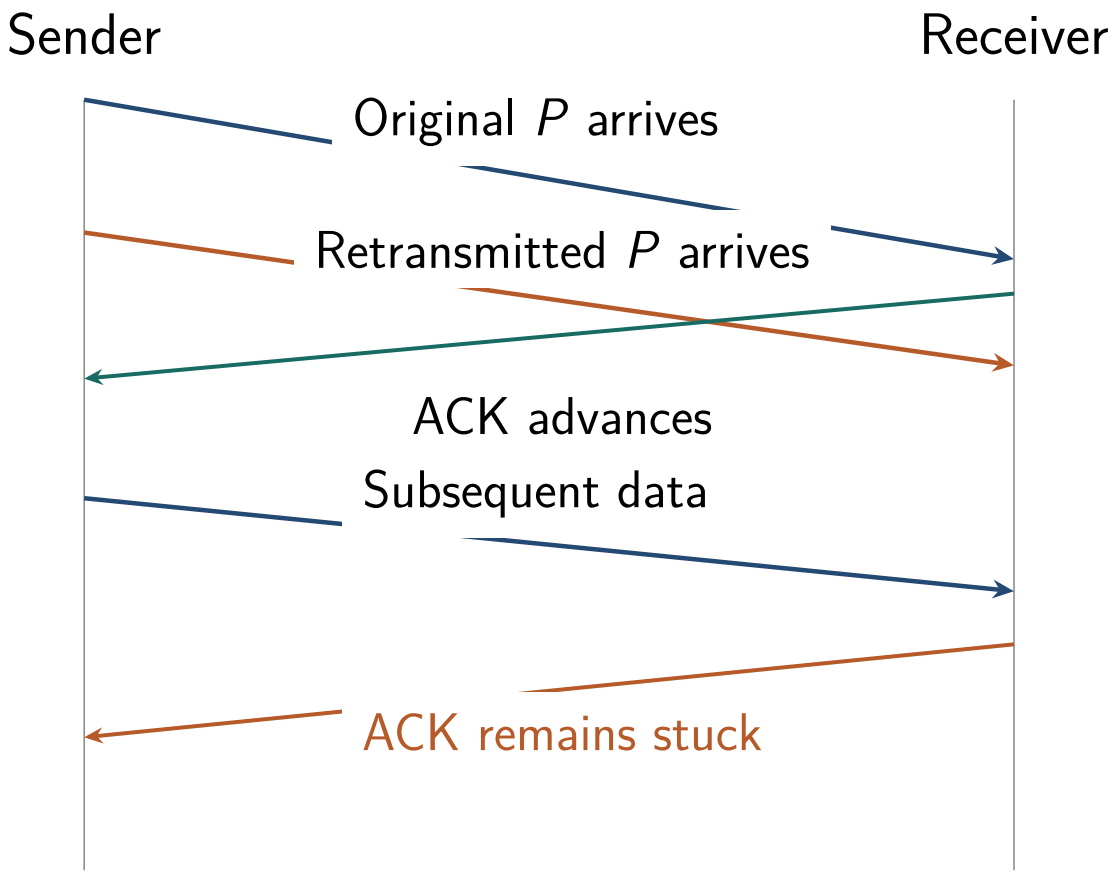
The receiver sends duplicate ACKs.

3. Three dup ACKs

The sender retransmits the delayed packet.

Sequence diagram by presenters, based on §4.4.1.

The arrival order triggers a receiver ACK bug



Original first, retransmission second

This order triggers the receiver's ACK-update logic bug.

Further data arrives, but the ACK number fails to advance.

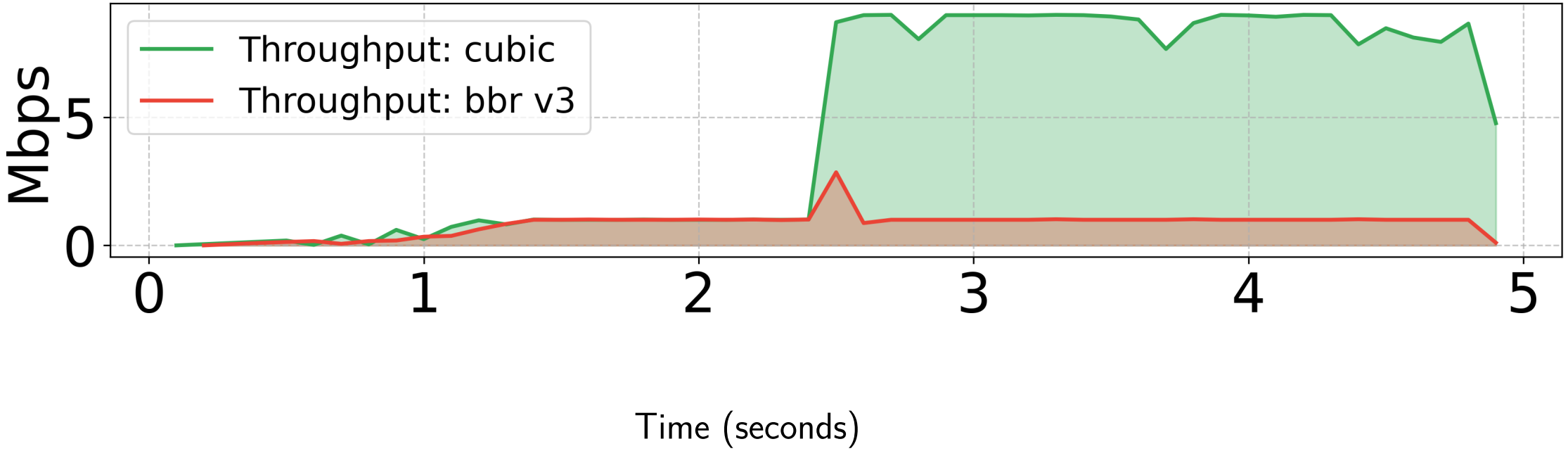
Repeated dup ACKs eventually force a retransmission timeout (RTO).

Diagnosis: eBPF confirmed the timeout; Wireshark confirmed packet arrival.

Sequence diagram by presenters, based on §4.4.1.

A newer kernel still exposes slow BBR growth

The earlier ACK bug was no longer observed on the newer kernel.



- CUBIC ramps up quickly in this discovered environment.
- BBR v3 underutilizes the path during the short run.

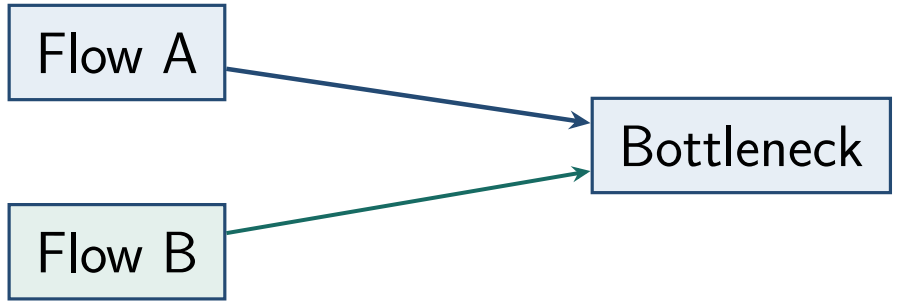
Fig. 6 (original throughput panel).

Either protocol can starve the other

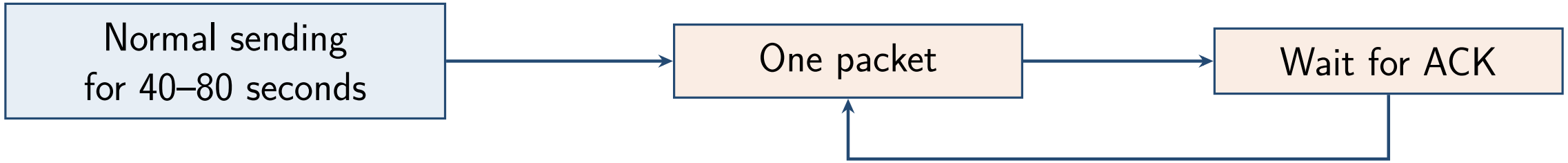
Reference: flow runs alone



Target: the same flow competes



CUBIC can starve BBR; other environments let BBR starve CUBIC.



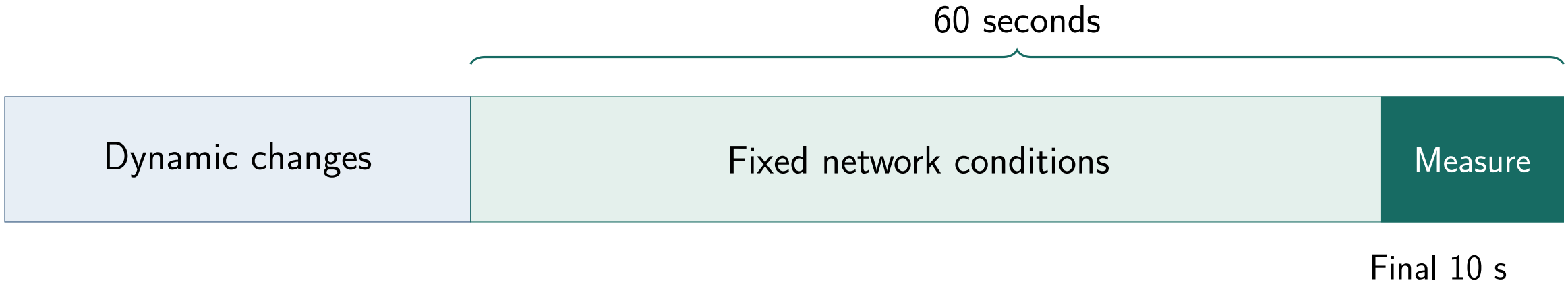
Regression testing measures the effect of a new version

Reference measurement

CUBIC throughput
while competing with BBR v1

Target measurement

CUBIC throughput
while competing with BBR v3



The final ten seconds are measured within the fixed one-minute period.

Experiment diagram by presenters, based on §4.4.5.

A version change can reverse the fairness outcome

CUBIC with BBR v1

12×

CUBIC's throughput
relative to BBR v1

CUBIC with BBR v3

11.6×

BBR v3's throughput
relative to CUBIC

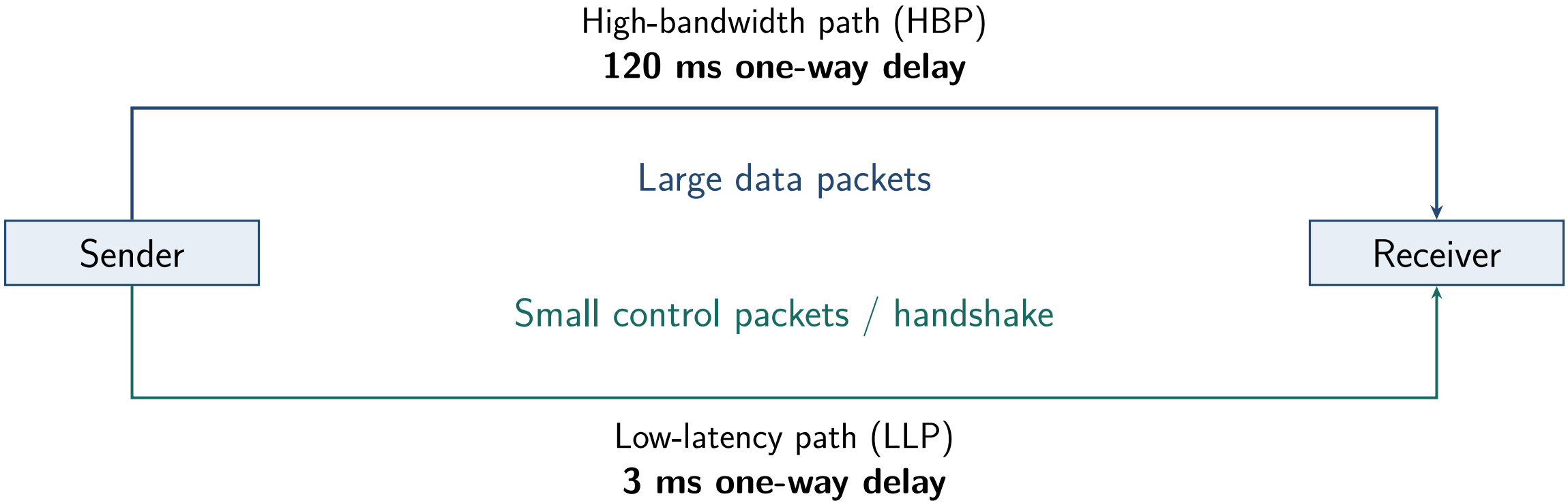
Single-flow throughput regressions also appear.

In another discovered environment, BBR v1 achieves **more than 5×**
the throughput of BBR v3.

Results from §4.4.5 and §1, arranged by presenters.

Can adding a path make performance worse?

Reference: high-bandwidth path alone **Target:** DChannel, two paths

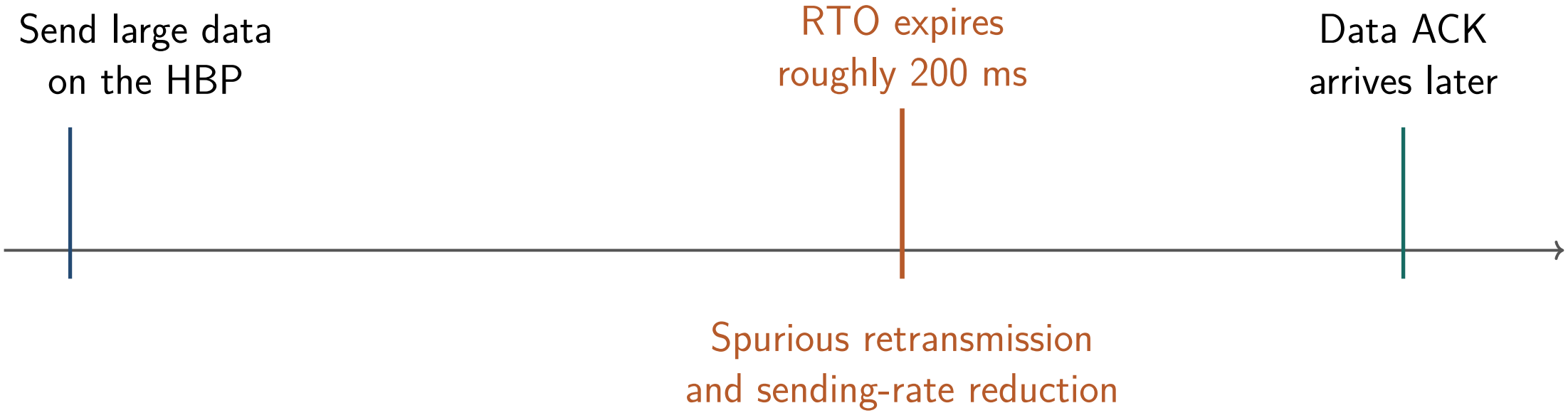


Discovered case: DChannel throughput falls to about one-third of HBP alone.

Path diagram by presenters, based on §4.4.3.

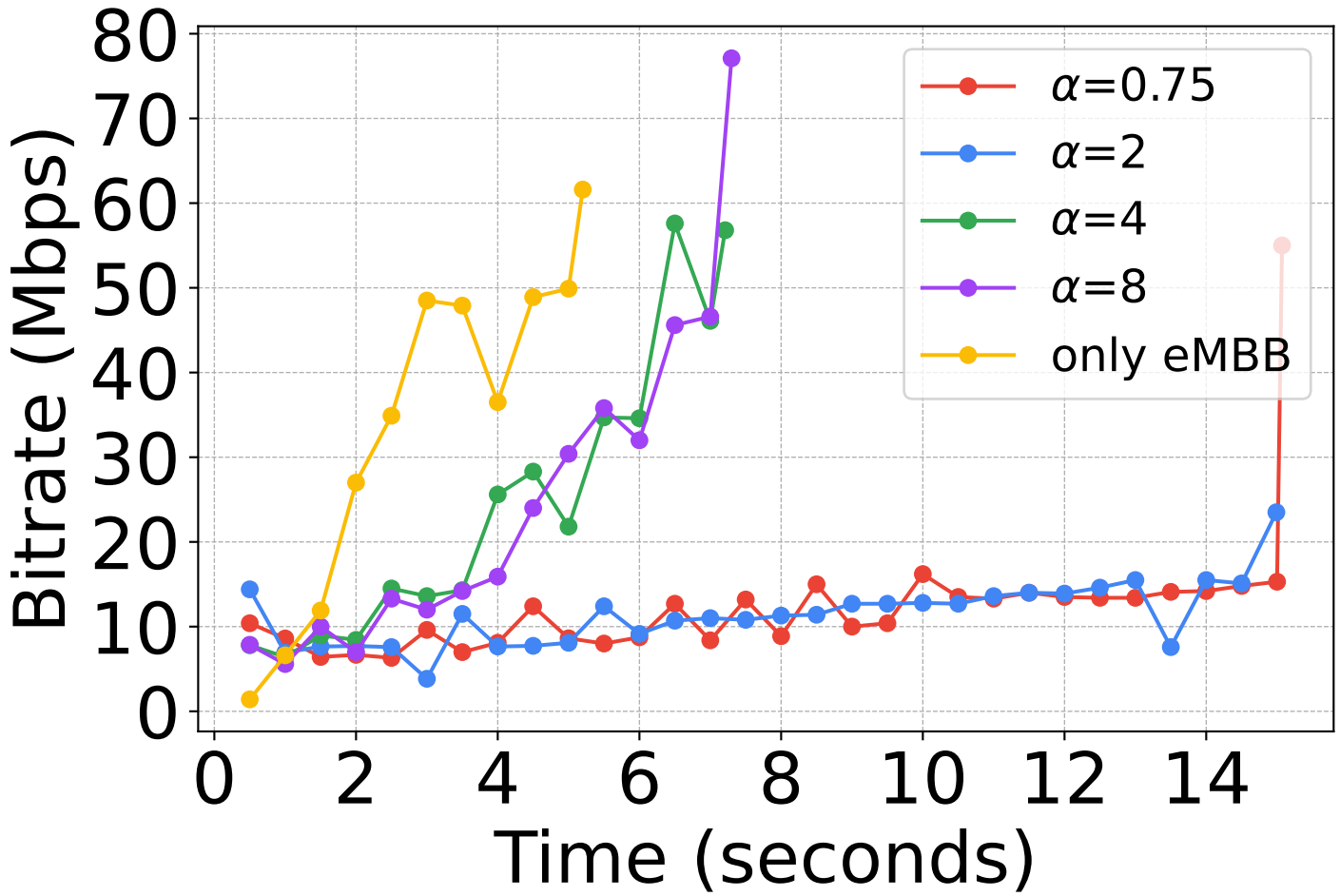
The handshake sets a timeout that data cannot meet

Handshake on the LLP → short RTT estimate → short RTO



Timing illustration by presenters, based on §4.4.3.

The single path finishes the transfer sooner



20 MB download

Earlier curve end = shorter completion time.

only eMBB: single HBP

Finishes before all tested DChannel α settings.

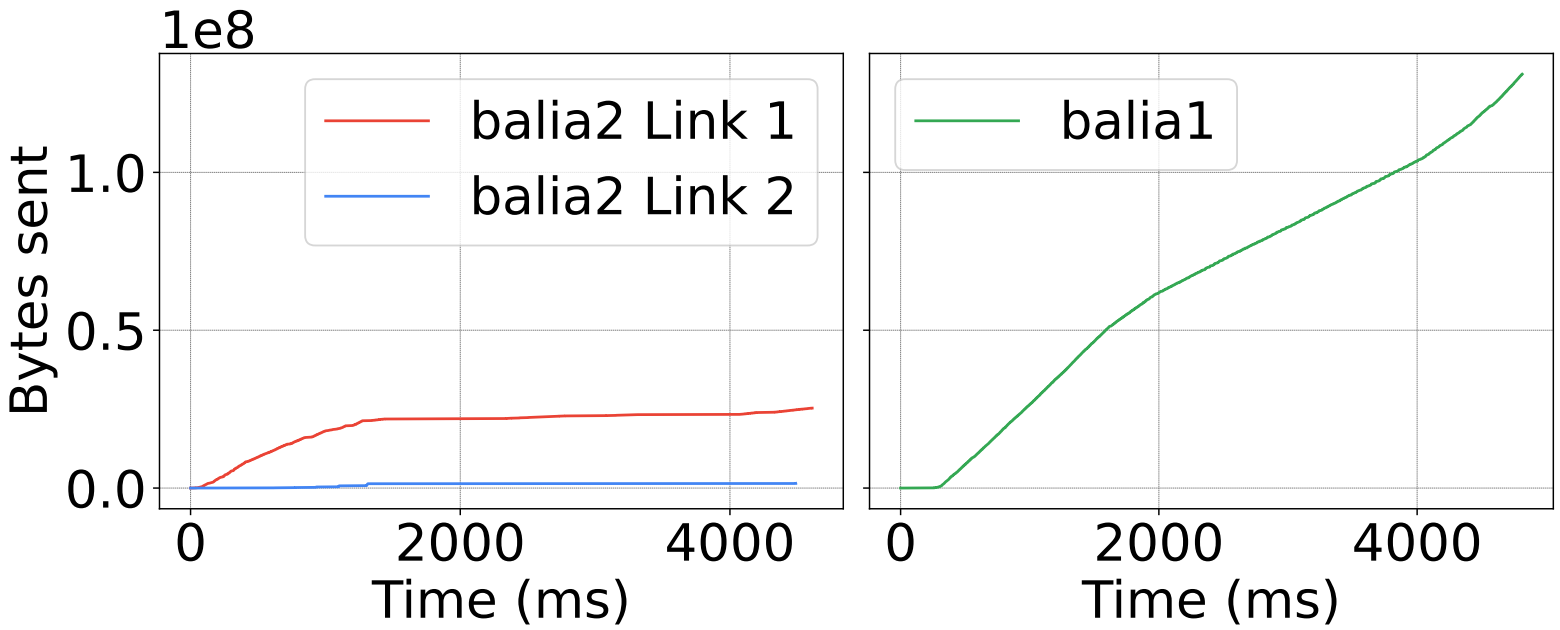
Reduce HBP RTT below 200 ms

DChannel regains its advantage over the single path.

Fig. 7 (original).

Kernel MPTCP also violates the two-path objective

Adversarial environments found for **LIA, OLIA, BALIA, and wVegas**.



About 0.5 s

Single-path catches up and overtakes dual-path.

About 1.2 s

A roughly 40 ms ACK delay precedes a sharp reduction on both links.

BALIA: dual-path throughput is 85.5% lower.

Fig. 8 (original).

One search pipeline supports three kinds of tests

Test	Reference	Target	What it reveals
UC1: weakness	Capacity or another protocol	Protocol under test	Underutilization and implementation bugs
UC2: regression	Older version	Newer version	Harmful effects of a change
UC3: design goal	One path	Two paths	Violations of a multipath objective

Choosing the reference turns a design question into an optimizable score.

Turn hidden weaknesses into reproducible test cases

AdvNet automates the search for performance problems in real protocol code.

Current scope

- Limited network parameters and simple traffic workloads
- Manual root-cause investigation
- No estimate of deployment frequency

Next steps

- Richer environments and workloads
- Automatic trace simplification
- Other domains: adaptive bitrate streaming prototype

Related approaches: formal verification and GANs

Formal verification

Checks specified properties using methods such as model checking and symbolic analysis.

Requires a precise model or specification; complex kernel code and changing networks create scale challenges.

Generative adversarial networks

Learn to generate data resembling a training dataset.

For AdvNet, a dataset of challenging environments is not already available.

AdvNet generates candidates using measured performance gaps as feedback.

Alternative post-learning selection methods

Method	Allocation rule	Selection idea
RoundRobin	Equal budget per candidate	Return the highest observed mean
OCBA	More runs for uncertain candidates and close contenders	Increase the probability of selecting the best true mean
MRE	Remove roughly the lower half each round; stop at ≤ 5	Concentrate measurements on promising candidates

OCBA: Optimal Computing Budget Allocation

GA operators in equations

Parents a, b; gene index i; cut points $k_1 < k_2$; per-gene bounds $[l_i, u_i]$.

Two-point crossover

$$c_i = \begin{cases} b_i & k_1 < i \le k_2, \\ a_i & \text{otherwise.} \end{cases}$$

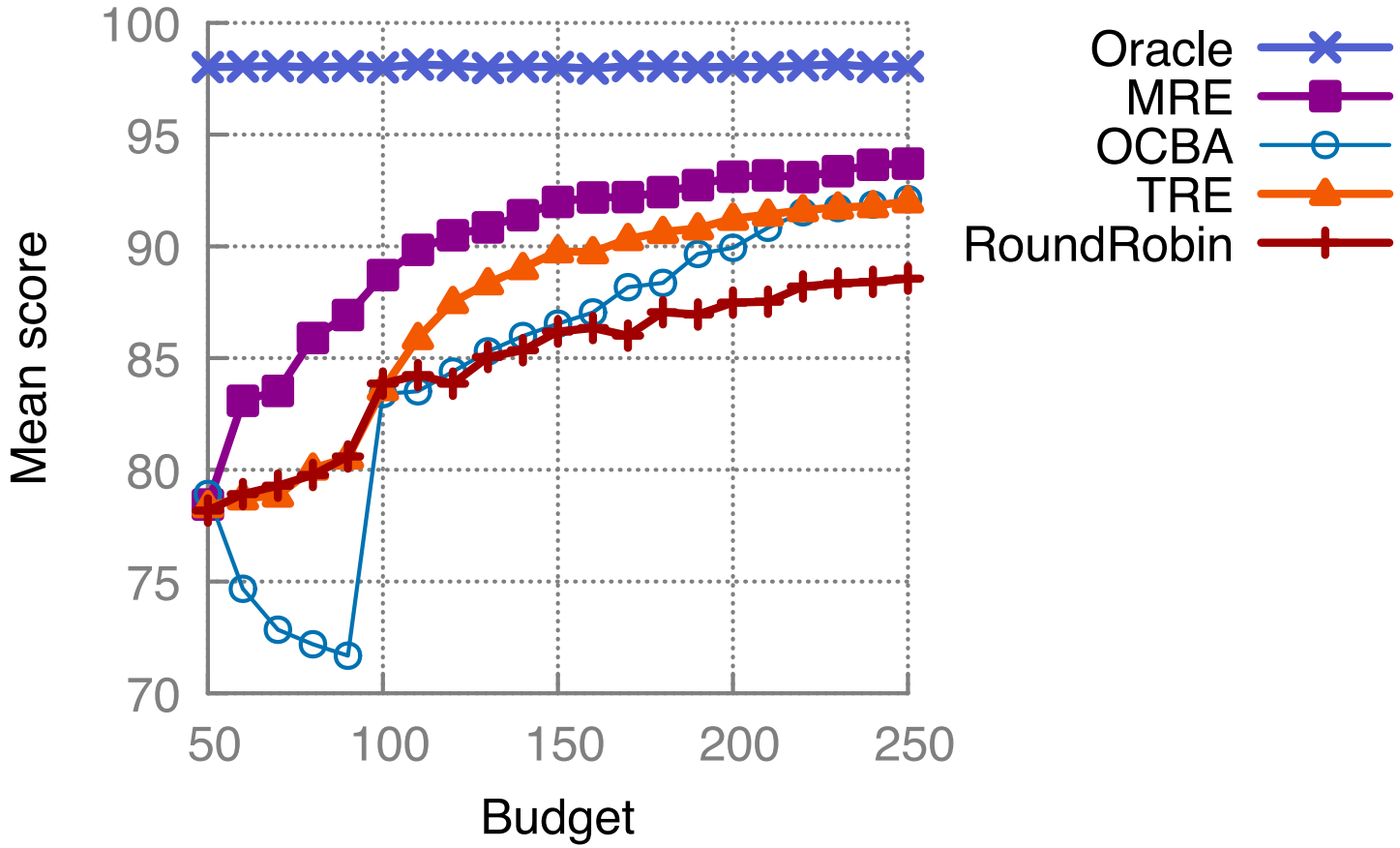
Uniform integer mutation

$$m_i = \begin{cases} U_i & \text{with probability } p, \\ c_i & \text{with probability } 1 - p, \end{cases} \qquad U_i \sim \text{UniformInteger}(l_i, u_i).$$

Operators from §3.2.2; notation by presenters.

Selection methods in a statistical simulation

Vertical axis: true mean of the selected alternative.



50 Gaussian alternatives

True means drawn uniformly from 0 to 100.

Results averaged over 2,000 trials.

MRE performs best.

Below 100 samples, OCBA has too few observations to estimate variance reliably.

Fig. 11 (original).